

Growing Hypergraphs with Node Labels

Francis Cataldo*,¹ Violet Ross*,² and Philip S. Chodrow¹

¹*Department of Computer Science, Middlebury College, Middlebury, VT, USA*

²*Department of Computer Science, University of Colorado at Boulder, Boulder, CO, USA*

(Dated: April 8, 2026)

We study a generative mechanistic model of hypergraph growth in which nodes possess binary labels which generalizes prior work on edge-copying growing hypergraphs. Edges form in this model as noisy copies of previous edges, where the transmission of nodes from one edge to the next depends on the labels. We derive a power law for the degree distribution in this model and describe the long-term dynamics of the joint distribution of labels contained in edges. We then demonstrate that this model can be learned from data: we can both estimate the parameters of the model via (stochastic) expectation maximization and perform community detection of the node labels via maximum-likelihood estimation.

I. INTRODUCTION

The use of a network to model a complex system is often carried out with one of two aims: modeling dynamics *on* a network or modeling the dynamics *of* a network. This work falls into the latter category — we model the growth of a complex system via the growth of its network representation. Specifically we focus on a type of complex system called a higher-order network and a type of network representation called a hypergraph.

Higher-order networks (HONs) are complex systems composed of entities and the multi-way relationships between them. Many real-world systems are composed of higher-order interactions. Social interaction (both in-person [1] and online [2]), academic co-authorship [3], and ecological systems [4] all contain interactions consisting of more than two entities.

It is often useful to represent an HON using a data structure that models its interactions. Hypergraphs, bipartite graphs, and simplicial complexes can all be used to model HONs [5]. Dyadic networks, or graphs, can be used to model dynamics on HONs when the equations of the dynamics properly account for multiway interaction [6]. However, when multiway interactions are not incorporated into the model specification, down-the-line analyses suffer [2, 7].

A hypergraph $\mathcal{H}$ is composed of a set of nodes $\mathcal{N}$ and a set of hyperedges between them $\mathcal{E}$. A hyperedge is an edge that can contain any number of nodes. We propose a model of HON growth using hypergraphs. The primary mechanism of our model is noisy copying of group interactions, or hyperedges. We therefore choose to formulate the approach as a hypergraph growth model in the interest of clear model description and analysis. Note, however, that this model could be equivalently formulated as a bipartite graph, simplicial complex, or annotated dyadic network growth model.

A. Higher order!!!

[5, 8, 9].

General hypergraph data mining:

[10]

B. Structural Properties of Higher-Order Network Data

[11]

[12]

Impact of overlap on dynamics: [13]

C. Generative Hypergraph Models

Higher-order intersections: [2]

CRU: [14]

Edge-copy model: [15]

THyME: [16]

A model based on diversity: [17]

D. Generative Models for Inference and Community Detection

Graph generative model for community detection: [18]

[19] [1, 20, 21]

[22]

E. Need to categorize

[23]

F. Tooling

PyTorch: [24]

Simulated annealing? [25]

Gradient descent [26]

* Equal author contributions

XGI!! [27]

II. LONG-RUN DYNAMICS

A. Joint Distribution of Edge Labels

We consider the joint distribution $p(k_0, k_1)$ of label counts in a uniformly random edge, where k_0 is the number of nodes with label 0 in the edge and k_1 is the number of nodes with label 1 in the edge. Although we are unable to find a closed-form expression for this distribution at stationarity, we can write down a linear map whose leading eigenvector corresponds to the stationary distribution. Let $p^{(t)}(k_0, k_1)$ be the joint distribution of label counts in a uniformly random edge after t timesteps. Then, the probability $q(k'_0, k'_1)$ of realizing an edge with k'_0 nodes of label 0 and k'_1 nodes of label 1 at time $t+1$ is given by

$$q(k'_0, k'_1) = \sum_{k_0, k_1} T(k'_0, k'_1 | k_0, k_1) p^{(t)}(k_0, k_1), \quad (1)$$

where $T(k'_0, k'_1 | k_0, k_1)$ gives the probability that the edge e realized in step $t+1$ has k'_0 nodes of label 0 and k'_1 nodes

of label 1 given that the edge f sampled to form e has k_0 nodes of label 0 and k_1 nodes of label 1. At stationarity, we must have $q(k'_0, k'_1) = p^{(t)}(k'_0, k'_1) = p^{(t+1)}(k'_0, k'_1)$, so the stationarity condition can be obtained by solving the linear system

$$p(k'_0, k'_1) = \sum_{k_0, k_1} T(k'_0, k'_1 | k_0, k_1) p(k_0, k_1). \quad (2)$$

The stationary joint distribution $p(k_0, k_1)$ is therefore given by the leading eigenvector of the doubly-indexed matrix T with entries $T(k'_0, k'_1 | k_0, k_1)$. The entries of this matrix can be computed analytically. Conditional on choosing an edge f with k_0 nodes of label 0 and k_1 nodes of label 1, a node of label z is selected as the seed node u with probability $k_z/(k_0 + k_1)$. Then, the number of nodes of labels z and $\bar{z}$ in e have distributions

$$\begin{aligned} k'_z &\sim 1 + \text{Binom}(k_z - 1, \eta_+) + \text{Poisson}(\lambda_+ + \gamma_+), \\ k'_{\bar{z}} &\sim \text{Binom}(k_{\bar{z}}, \eta_-) + \text{Poisson}(\lambda_- + \gamma_-). \end{aligned}$$

The probability of obtaining k'_0 nodes of label 0, conditioning on k_0 and k_1 , is

$$t(k'_0 | k_0, k_1) = \frac{k_0}{k_0 + k_1} \sum_{i=0}^{k'_0-1} \binom{k_0-1}{i} \eta_+^i (1-\eta_+)^{k_0-1-i} e^{-(\lambda_++\gamma_+)} \frac{(\lambda_++\gamma_+)^{k'_0-1-i}}{(k'_0-1-i)!} + \quad (3)$$

$$\frac{k_1}{k_0 + k_1} \sum_{i=0}^{k'_0} \binom{k_0}{i} \eta_-^i (1-\eta_-)^{k_0-i} e^{-(\lambda_-+\gamma_-)} \frac{(\lambda_-+\gamma_-)^{k'_0-i}}{(k'_0-i)!}. \quad (4)$$

A parallel expression describes $t(k'_1 | k_0, k_1)$, and the transition matrix entries are given by the product

$$T(k'_0, k'_1 | k_0, k_1) = t(k'_0 | k_0, k_1) t(k'_1 | k_0, k_1). \quad (5)$$

In Figure 2, we show projections of the joint distribution $p^{(t)}(k_0, k_1)$ onto the top three eigenvectors of the matrix T during a simulation run. These eigenvectors are computed numerically. The Perron eigenvector v_1 with eigenvalue 1 corresponds to the stationary distribution $p(k_0, k_1)$, while the second and third eigenvectors v_2 and v_3 correspond to transient dynamics. The corresponding eigenvalues are relatively large, indicating that these transients decay relatively slowly.

B. Mean Degree

The mean degree $\langle d \rangle$ of a node satisfies

$$\langle d \rangle = \frac{m}{n} \langle k \rangle, \quad (6)$$

where $\langle k \rangle$ is the mean edge size, m is the number of edges, and n is the number of nodes. In expectation, there are $\gamma_- + \gamma_+$ new nodes added per new edge, which means that $\frac{m}{n} \rightarrow \frac{1}{\gamma_- + \gamma_+}$ as $m, n \rightarrow \infty$ in expectation. So, we have

$$\langle d \rangle = \frac{\langle k \rangle}{\gamma_- + \gamma_+}. \quad (7)$$

We don't have a closed-form expression for the stationary value of $\langle k \rangle$, but we can compute it numerically from the stationary distribution of the joint label counts $p(k_0, k_1)$.

C. Power Law Degree Distribution

This argument follows that of [15], which in turn is based on a derivation by [28].

Under the mean-field assumption that the degree of a node is independent of the ratio $\frac{k_0}{k}$ of the edge it is contained in, we can write down an argument to show

that the degree distribution follows a power law and give a formula for the exponent. Define the moments

$$\rho_0 = \left\langle \frac{k_0^2}{k} \right\rangle, \quad \rho_1 = \left\langle \frac{k_1^2}{k} \right\rangle \quad \text{and} \quad \rho_{01} = \left\langle \frac{k_0 k_1}{k} \right\rangle. \quad (8)$$

Then, the expected number of nodes which are sampled in the edge-sampling step is

$$\sigma \triangleq 1 + \eta_+ [\rho_0 + \rho_1 - 1] + \eta_- \rho_{01}. \quad (9)$$

Importantly, σ depends only on the parameters and the moments of the joint distribution $p(k_0, k_1)$, which we are

going to show how to get from linear algebra later. Let $\tilde{\sigma} = \frac{\sigma}{\langle d \rangle}$, where $\langle d \rangle$ is the average degree of a node in the hypergraph.

Under a mean-field assumption in which the degree d of a node is independent from the counts k_0 and k_1 of labels in an edge containing that node, we can give the expected change in the count of degree d nodes as

$$\tilde{\sigma}((d-1)p_{d-1}^{(t)} - dp_d^{(t)}) \quad (10)$$

Similarly, the expected number of nodes which are sampled in the extant node addition step is $\lambda_+ + \lambda_-$. This gives, in expectation for the change in degree d nodes,

$$n^{(t+1)}p_d^{(t+1)} - n^{(t)}p_d^{(t)} = \tilde{\sigma}[(d-1)p_{d-1}^{(t)} - dp_d^{(t)}] + (\lambda_+ + \lambda_-)(p_{d-1}^{(t)} - p_d^{(t)}). \quad (11)$$

We know that at stationarity, $n^{(t+1)} - n^{(t)}$ is in expectation equal to $\gamma_+ + \gamma_-$, while at stationarity $p_d^{(t+1)} = p_d^{(t)} = p_d$. Thus, we obtain

$$p_d(\gamma_+ + \gamma_-) = \tilde{\sigma}[(d-1)p_{d-1} - dp_d] + (\lambda_+ + \lambda_-)(p_{d-1} - p_d). \quad (12)$$

Rearranging, we obtain

$$\frac{p_d}{p_{d-1}} = \frac{\tilde{\sigma}(d-1) + \lambda_+ + \lambda_-}{\tilde{\sigma}d + \gamma_+ + \gamma_- + \lambda_+ + \lambda_-} \quad (13)$$

$$= 1 - \frac{\tilde{\sigma} + \gamma_+ + \gamma_-}{\tilde{\sigma}d + \gamma_+ + \gamma_- + \lambda_+ + \lambda_-} \quad (14)$$

Allowing d to become large (which describes the tail of the degree distribution), we have

$$\frac{p_d}{p_{d-1}} \approx 1 - \frac{\tilde{\sigma} + \gamma_+ + \gamma_-}{\tilde{\sigma}d}, \quad (15)$$

a relation which corresponds to a power law with exponent $\zeta = 1 + \frac{\gamma_+ + \gamma_-}{\tilde{\sigma}}$. Explicitly calculating this moment requires computing the moments ρ_0 , ρ_1 , and ρ_{01} , which can be done computationally from the stationary joint distribution $p(k_0, k_1)$.

D. Joint Distribution of Labels

III. MODEL INFERENCE

PC: Someone needs to establish standardized notation in this section.

A. Parameter Estimation via Stochastic EM

PC: Violet, this is your spot! Descriptions of methods/algorithms should go here, results are for later.

We introduce a Stochastic Expectation Maximization (SEM) algorithm for inferring the [INSERT MODEL NAME HERE] model parameters from a hypergraph. A traditional inference approach, maximum likelihood estimation (MLE), infers parameter values by selecting the set of parameters that are most likely to have generated the observed data. The probability that our data, the hypergraph $\mathcal{H}$, was generated using a given set of parameters θ depends not only on observable data (the hypergraph itself) but also on latent variables (the edge e' and node u selected at each timestep of the growth model). Therefore, to compute the log marginal likelihood, which MLE seeks to maximize over, we must marginalize over the latent variables, making this approach computationally expensive. Expectation Maximization (EM) algorithms also maximize over the log marginal likelihood, but avoid expensive marginalization. However, the original EM algorithm [29] computes the expected value of a sufficient statistic using all observations at each time step, which introduces another computational cost. Stochastic Expectation Maximization [30] resolves this issue by, at each timestep, computing the expectation using only one randomly selected data



FIG. 1. Degree distribution from simulation (points) and theoretical power law (line) with exponent computed from model parameters and joint label distribution moments. **PC:** Put three series on this plot and give slopes for each. Also, fairly sure the calculation is bugged in either math or code.

point.

Each iteration of the SEM algorithm consists of two steps: the E-step, in which the expected values of the sufficient statistics are calculated using the current parameter estimates, and the M-step, in which the parameter estimates are updated using the expected sufficient statistics.

Suppose we have an observed variable x and a hidden variable h . Let $f(x_i, h_i)$ be a function which computes a sufficient statistic for each of the parameters in θ and let $F_i(\hat{\theta}) = \mathbb{E}[f(x_i, h_i)|x_i]$ be the function which computes the expected sufficient statistic values given an observed x_i . Let g be a function which computes the maximum likelihood estimate of θ from the sufficient statistics. Fix a sequence of learning rates $\ell^{(1)}, \dots, \ell^{(t)}$. Each iteration, t , of SEM proceeds as follows.

E step: Sample an observation x_i from the data uniformly at random. Compute the expected sufficient statistics $F_i(\hat{\theta}^{(t-1)})$ using the randomly selected data point x_i and the parameter estimates $\hat{\theta}^{(t-1)}$ from the previous timestep. Generate an new estimate of sufficient statistics $s^{(t)}$ according to the update rule: $s^{(t)} = (1 - \ell^{(t)})s^{(t-1)} + \ell^{(t)}F_i(\hat{\theta}^{(t-1)})$.

M step: Compute the new parameter estimates $\hat{\theta}^{(t)}$ from the updated sufficient statistics: $\hat{\theta}^{(t)} = g(s^{(t)})$.

1. Description of SEM on hypergraphs

We use an SEM algorithm to estimate the parameters $\theta = (p, q, \gamma_{e,z_u}, \gamma_{e,\bar{z}_u}, \gamma_{e,z_n}, \gamma_{e,\bar{z}_n})$ used to generate a hypergraph $\mathcal{H}$ via the [MODEL NAME] model. At each timestep, we have an observed variable e' (a hyperedge) and two hidden variables u and e (the node and the hyperedge that generated e'). Let f be a function which computes a sufficient statistic for each of the parameters in θ . Specifically, let

$$f(e, u, e') = (t_{z_u}, w_{z_u}, t_{\bar{z}_u}, w_{\bar{z}_u}, n_{e,z_u}, n_{e,\bar{z}_u}, n_{n,z_u}, n_{n,\bar{z}_u})$$

where w_{z_u} is the number of nodes with label z_u in e , $w_{\bar{z}_u}$ is the number of nodes with the label $\bar{z}_r$ in e , t_{z_u} and $t_{\bar{z}_u}$ are the numbers of nodes with label z_u and $\bar{z}_r$, respectively, copied from e to e' , n_{e,z_u} and $n_{e,\bar{z}_u}$ are the numbers of external nodes with labels z_u and $\bar{z}_r$ in e' , and n_{n,z_u} and $n_{n,\bar{z}_u}$ are the numbers of new nodes with labels z_u and $\bar{z}_r$ in e' . The sufficiency of these statistics follows from standard results [?]. Then the function

$$F_{e'}(\hat{\theta}^{(t-1)}) = \mathbb{E}[f(e, u, e')|e', \hat{\theta}^{(t-1)}] \\ = \frac{\sum_{e \in \mathcal{E}_{\text{prev}}} \sum_{u \in e \cap e'} p(e, u, e'|\hat{\theta}^{(t-1)}) f(e, u, e')}{\sum_{e \in \mathcal{E}_{\text{prev}}} \sum_{u \in e \cap e'} p(e, u, e'|\hat{\theta}^{(t-1)})}$$

computes the expected values of the sufficient statistics at time t given edge e' . Finally, the maximum likelihood estimate of θ is given by

$$g(s^{(t)}) = \left(\frac{t_u}{w_{z_u}}, \frac{t_{\bar{z}_u}}{w_{\bar{z}_u}}, n_{e,z_u}, n_{e,\bar{z}_u}, n_{n,z_u}, n_{n,\bar{z}_u} \right).$$

Each iteration of our SEM algorithm for hypergraphs then proceeds following the E- and M-steps detailed above.

B. Community Detection via MLE

PC: Frannie, this is your spot! Descriptions of methods/algorithms should go here, results are for later.

IV. INFERENCE RESULTS

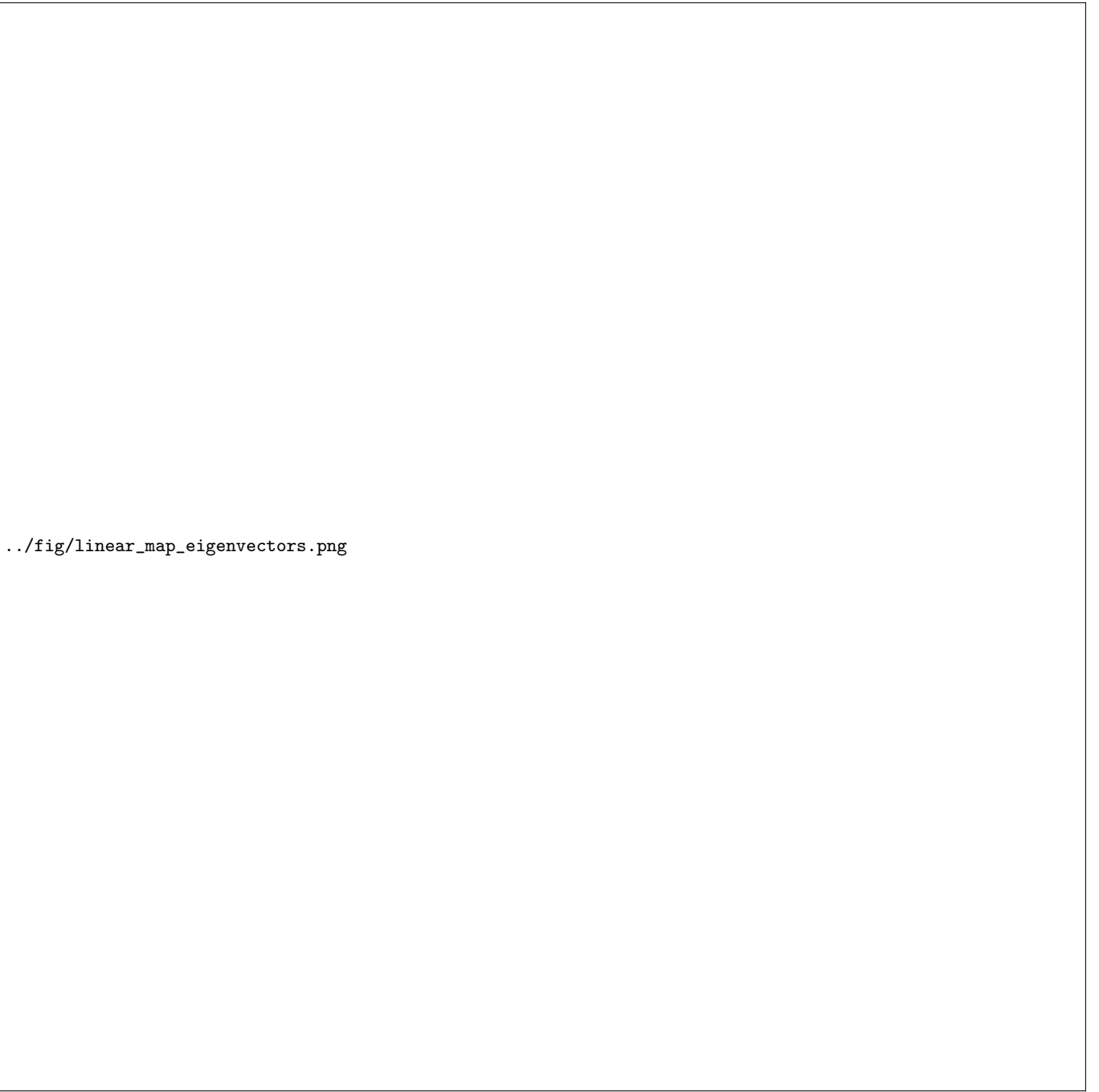
How did we do on real and synthetic data?

V. CONCLUSION

Blah blah blah.

ACKNOWLEDGMENTS

PSC acknowledges support from the National Science Foundation under DMS-2407058.



../fig/linear_map_eigenvectors.png

FIG. 2. Dynamics! Top shows the three leading eigenvectors of the distribution $p(k_0, k_1)$. Bottom left gives the cosine similarity (normalized projection) between one run of simulation dynamics and these eigenvectors. Bottom right gives the final simulation state. **PC: Need to check these projections, looks a bit strange. Can run 10^7 steps in two minutes on laptop.**

-
- [1] P. S. Chodrow, N. Veldt, and A. R. Benson, *Science Advances* **7**, eabh1303 (2021).
 - [2] A. R. Benson, R. Abebe, M. T. Schaub, A. Jadbabaie, and J. Kleinberg, *Proceedings of the National Academy of Sciences* **115**, E11221 (2018).
 - [3] H. Wu, Y. Yan, and M. K.-P. Ng, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **45**, 3245 (2022).
 - [4] J. M. Levine, J. Bascompte, P. B. Adler, and S. Allesina, *Nature* **546**, 56 (2017).

- [5] F. Battiston, G. Cencetti, I. Iacopini, V. Latora, M. Lucas, A. Patania, J.-G. Young, and G. Petri, *Physics Reports* **874**, 1 (2020).
- [6] T. P. Peixoto, L. Peel, T. Gross, and M. De Domenico, arXiv preprint arXiv:2602.16937 (2026).
- [7] S. Klamt, U.-U. Haus, and F. Theis, *PLoS computational biology* **5**, e1000385 (2009).
- [8] L. Torres, A. S. Blevins, D. Bassett, and T. Eliassi-Rad, *SIAM Review* **63**, 435 (2021).
- [9] F. Battiston, E. Amico, A. Barrat, G. Bianconi, G. Ferraz de Arruda, B. Franceschiello, I. Iacopini, S. Kéfi, V. Latora, Y. Moreno, *et al.*, *Nature Physics* **17**, 1093 (2021).
- [10] G. Lee, F. Bu, T. Eliassi-Rad, and K. Shin, *ACM Computing Surveys* **57**, 1 (2025).
- [11] N. W. Landry, J.-G. Young, and N. Eikmeier, *EPJ Data Science* **13**, 17 (2024).
- [12] G. Lee, M. Choe, and K. Shin, in *Proceedings of the Web Conference 2021* (ACM, Ljubljana Slovenia, 2021) pp. 3396–3407.
- [13] F. Malizia, S. Lamata-Otín, M. Frasca, V. Latora, and J. Gómez-Gardeñes, *Nature Communications* **16**, 555 (2025).
- [14] A. R. Benson, R. Kumar, and A. Tomkins, in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (ACM, London United Kingdom, 2018) pp. 1148–1157.
- [15] X. He, P. S. Chodrow, and P. J. Mucha, *Physical Review E* **112**, 024305 (2025).
- [16] G. Lee and K. Shin, in *2021 IEEE International Conference on Data Mining (ICDM)* (IEEE, Auckland, New Zealand, 2021) pp. 310–319.
- [17] X. Yu and J. Zhu, *Journal of the American Statistical Association* **120**, 1491 (2025).
- [18] B. Karrer and M. E. J. Newman, *Phys. Rev. E* **83**, 016107 (2011).
- [19] N. Ruggeri, M. Contisciani, F. Battiston, and C. De Bacco, *Science Advances* **9**, eadg9159 (2023).
- [20] J. Kritschgau, D. Kaiser, O. Alvarado Rodriguez, I. Amburg, J. Bolkema, T. Grubb, F. Lan, S. Maleki, P. S. Chodrow, and B. Kay, *Scientific Reports* **14**, 6933 (2024).
- [21] P. S. Chodrow, N. Eikmeier, and J. L. Haddock, *SIAM Journal on Mathematics of Data Science* **5**, 251 (2023).
- [22] J. Hood, C. D. Bacco, and A. Schein, *Broad Spectrum Structure Discovery in Large-Scale Higher-Order Networks* (2025), arXiv:2505.21748 [cs].
- [23] A. Badalyan, N. Ruggeri, and C. De Bacco, *Nature Communications* **15**, 7073 (2024).
- [24] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, in *Proceedings of the 33rd International Conference on Neural Information Processing Systems* (Curran Associates Inc., Red Hook, NY, USA, 2019).
- [25] S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi, *Science* **220**, 671 (1983).
- [26] S. Ruder, *ArXiv abs/1609.04747* (2016).
- [27] N. W. Landry, M. Lucas, I. Iacopini, G. Petri, A. Schwarze, A. Patania, and L. Torres, *Journal of Open Source Software* **8**, 5162 (2023).
- [28] M. Mitzenmacher, *Internet Mathematics* **1**, 226 (2004).
- [29] A. P. Dempster, N. M. Laird, and D. B. Rubin, *Journal of the Royal Statistical Society: Series B (Methodological)* **39**, 1 (1977).
- [30] O. Cappé and E. Moulines, *Journal of the Royal Statistical Society Series B: Statistical Methodology* **71**, 593 (2009).